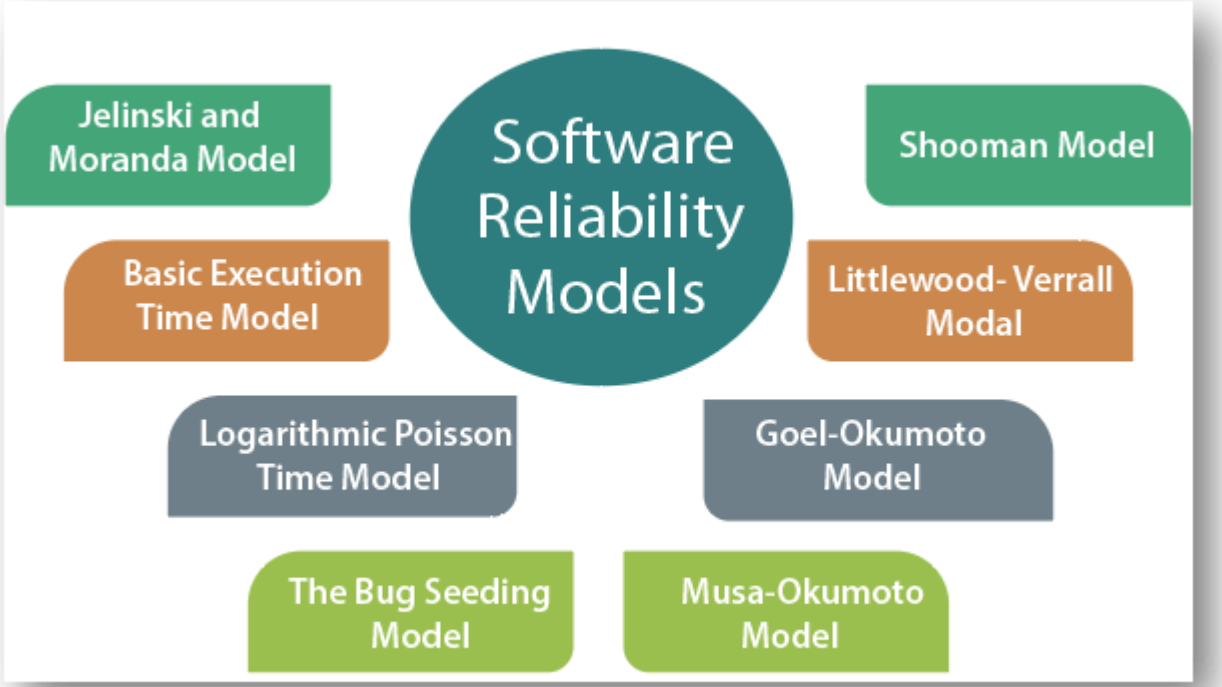
Software Reliability Model

- In Reliability Models , our emphasis is on failure rather than faults.
- Software Reliability Model specifies the general form of the dependence of the failure process on the factors mentioned.
- The Reliability of a program increase through fault correction and have the failure intensity decreases.

Failure VS Faults

- A faults is the defect in the program that when executed causes a failure.
- The program must execute for a failure to occur.
- A fault can be source of more than one failure.
- Fault is mostly created due to error by programmer.
- Failure is the inability of a system or a component to perform required function according to its specification.

Types of Software Reliability Model



Basic Execution Time Model

- This model was established by J.D. Musa in 1979, and it is based on execution time.
- The basic execution model is the most popular and generally used reliability growth model, mainly because:

  - It is practical, simple, and easy to understand.
  - Its parameters clearly relate to the physical world.
  - It can be used for accurate reliability prediction.

- The basic execution model determines failure behaviour initially using execution time. Execution time may later be converted in calendar time.

- Variables involved in the Basic Execution Model:
  - Failure intensity (λ): number of failures per time unit.
  - Execution time (τ): time since the program is running.
  - Mean failures experienced (μ): mean failures experienced in a time interval.
  - (Vo) It is a number of failure experienced , If program is run for infinite time period.

- Basic Execution Time Model

Current failure intensity

$$\lambda(\mu) = \lambda_0\left(1 - \frac{\mu}{V_0}\right) \qquad (1)$$

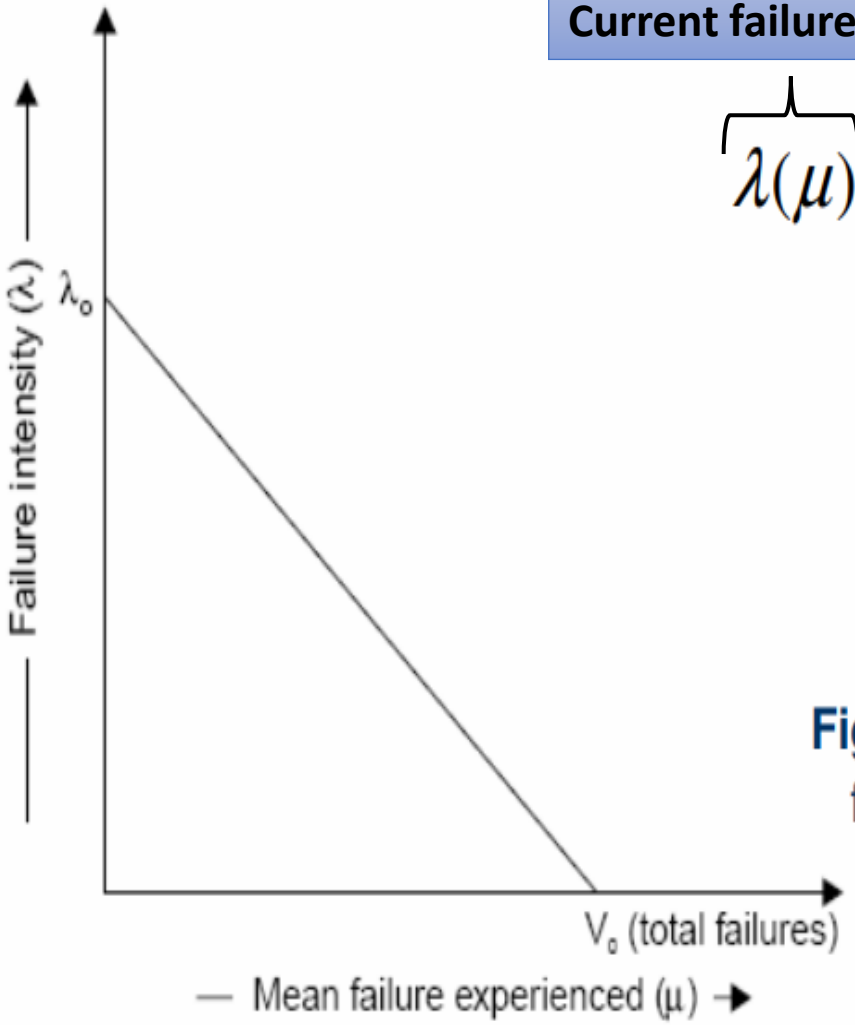
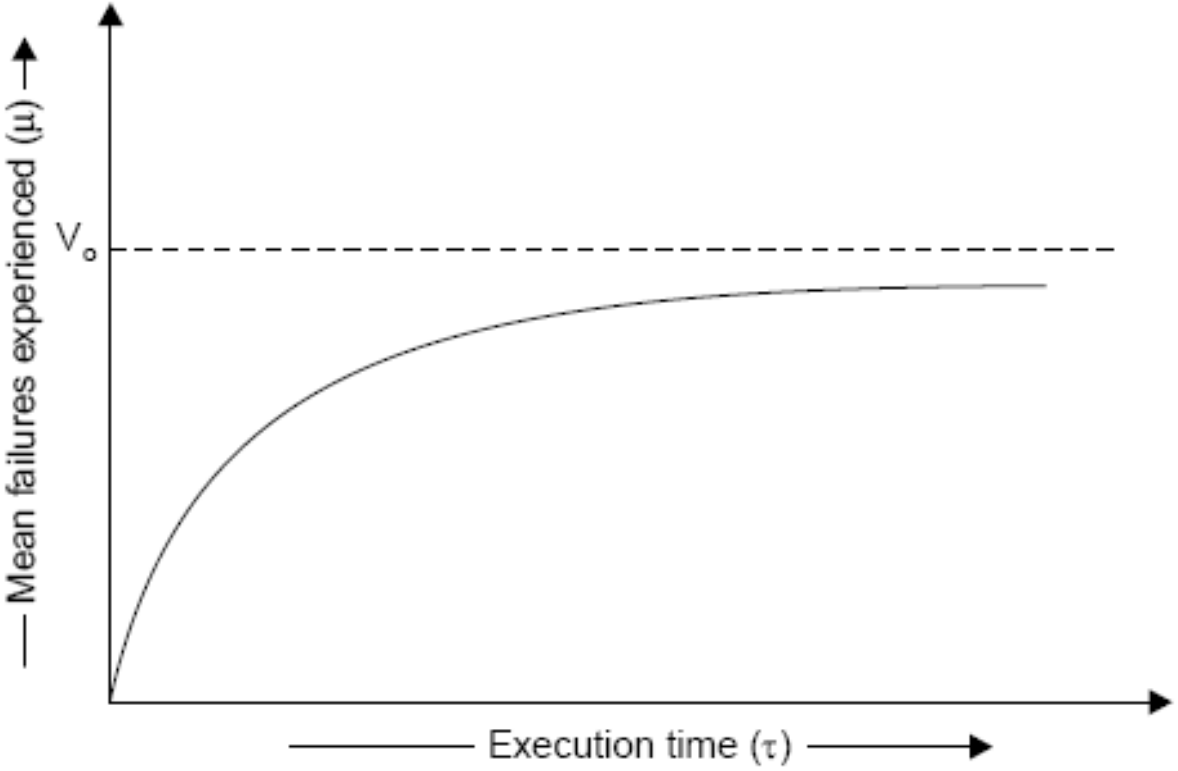


Fig.7.13: Failure intensity λ as a function of μ for basic model



Decrement of Failure Intensity per Failure

$$\frac{d\lambda}{d\mu} = \frac{-\lambda_0}{V_0} \qquad (2)$$

Relationship between τ & μ for basic model

Software Reliability

For a derivation of this relationship, equation 1 can be written as:

$$\frac{d\mu(\tau)}{d\tau} = \lambda_0\left(1 - \frac{\mu(\tau)}{V_0}\right)$$

The above equation can be solved for $\mu(\tau)$ and result in :

Failure Experienced after τ CPU hrs. of execution

$$\mu(\tau) = V_0\left(1 - \exp\left(\frac{-\lambda_0\tau}{V_0}\right)\right) \quad (3)$$

Software Reliability

The failure intensity as a function of execution time is shown in figure given below

Failure Intensity after τ CPU hrs. of execution

$$\lambda(\tau) = \lambda_0 \exp\left(\frac{-\lambda_0 \tau}{V_0}\right)$$

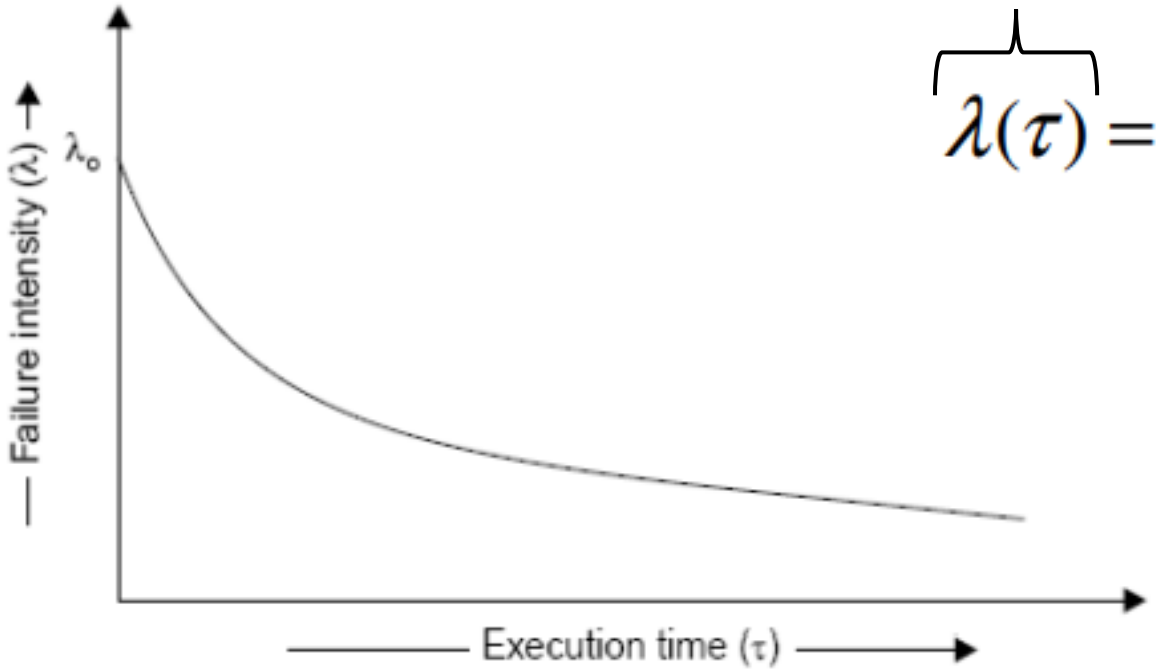
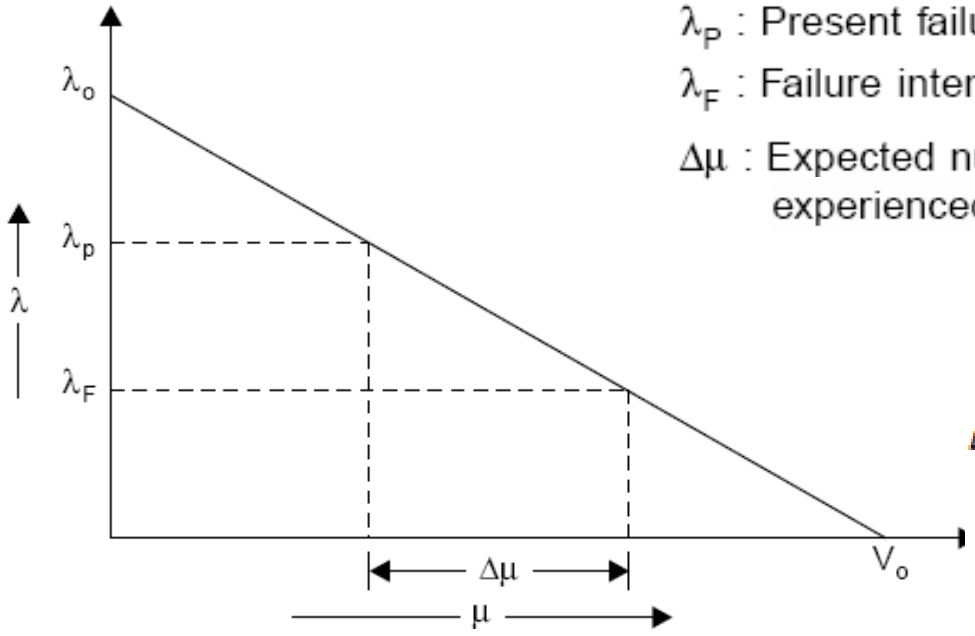


Fig.7.15: Failure intensity versus execution time for basic model

- Derived quantities



λ_0 : Initial failure intensity

λ_P : Present failure intensity

λ_F : Failure intensity objective

$\Delta\mu$: Expected number of additional failures to be experienced to reach failure intensity objective.

$$\Delta\mu = \left(\frac{V_0}{\lambda_0}\right)(\lambda_P - \lambda_F)$$

Additional failures required to be experienced to reach the objective

Software Reliability

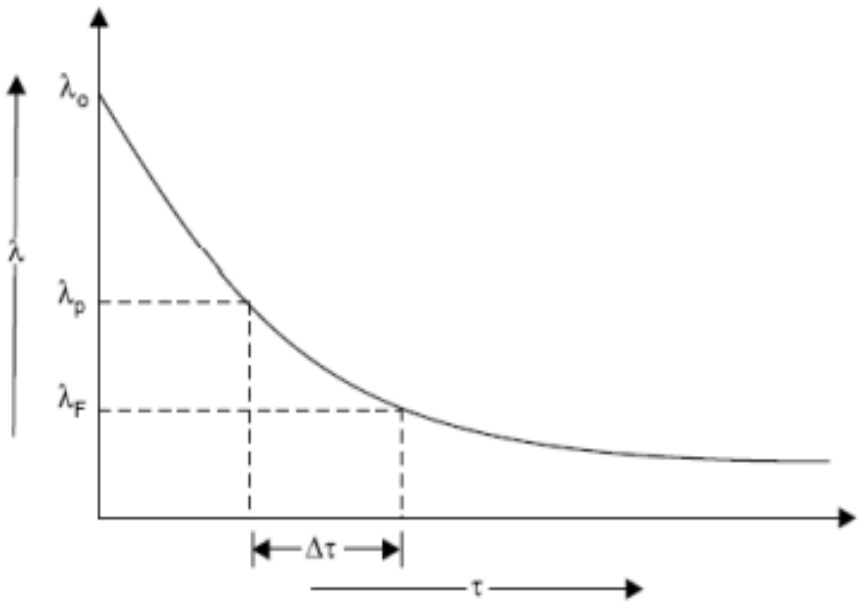


Fig.7.17: Additional time required to reach the objective

This can be derived in mathematical form as:

$$\Delta\tau = \frac{V_0}{\lambda_0} Ln\left(\frac{\lambda_P}{\lambda_F}\right)$$

Example- 7.1

Assume that a program will experience 200 failures in infinite time. It has now experienced 100. The initial failure intensity was 20 failures/CPU hr.

(i) Determine the current failure intensity.

(ii) Find the decrement of failure intensity per failure.

(iii) Calculate the failures experienced and failure intensity after 20 and 100 CPU hrs. of execution.

(iv) Compute addition failures and additional execution time required to reach the failure intensity objective of 5 failures/CPU hr.

Use the basic execution time model for the above mentioned calculations.

Software Reliability

Solution

Here V_0=200 failures

$$\mu = 100 \text{ failures}$$

$$\lambda_0 = 20 \text{ failures/CPU hr.}$$

(i) Current failure intensity:

$$\lambda(\mu) = \lambda_0\left(1 - \frac{\mu}{V_0}\right)$$

$$= 20\left(1 - \frac{100}{200}\right) = 20(1 - 0.5) = 10 \text{ failures/CPU hr}$$

Software Reliability

(ii) Decrement of failure intensity per failure can be calculated as:

$$\frac{d\lambda}{d\mu} = \frac{-\lambda_0}{V_0} = -\frac{20}{200} = -0.1/\text{CPU hr.}$$

(iii) (a) Failures experienced & failure intensity after 20 CPU hr:

$$\mu(\tau) = V_0\left(1 - \exp\left(\frac{-\lambda_0\tau}{V_0}\right)\right)$$

$$= 200\left(1 - \exp\left(\frac{-20\times 20}{200}\right)\right) = 200(1 - \exp(1-2))$$

$$= 200(1 - 0.1353) \approx 173\text{failures}$$

Software Reliability

$$\lambda(\tau) = \lambda_0 \exp\left(\frac{-\lambda_0 \tau}{V_0}\right)$$

$$= 20 \exp\left(\frac{-20 \times 20}{200}\right) = 20 \exp(-2) = 2.71 \, failures / CPU \ hr$$

(b) Failures experienced & failure intensity after 100 CPU hr:

$$\mu(\tau) = V_0 \left(1 - \exp\left(\frac{-\lambda_0 \tau}{V_0}\right)\right)$$

$$= 200 \left(1 - \exp\left(\frac{-20 \times 100}{200}\right)\right) = 200 \text{ failures(almost)}$$

$$\lambda(\tau) = \lambda_0 \exp\left(\frac{-\lambda_0 \tau}{V_0}\right)$$

Software Reliability

$$= 20\exp\left(\frac{-20\times 100}{200}\right) = 0.000908 \; failures / CPU \; hr$$

(iv) Additional failures $(\Delta\mu)$ required to reach the failure intensity objective of 5 failures/CPU hr.

$$\Delta\mu = \left(\frac{V_0}{\lambda_0}\right)(\lambda_P - \lambda_F) = \left(\frac{200}{20}\right)(10-5) = 50 \text{ failures}$$

Software Reliability

Additional execution time required to reach failure intensity objective of 5 failures/CPU hr.

$$\Delta\tau = \left(\frac{V_0}{\lambda_0}\right) Ln\left(\frac{\lambda_P}{\lambda_F}\right)$$

$$= \frac{200}{20} Ln\left(\frac{10}{5}\right) = 6.93 \text{ CPU hr.}$$